

Lab 07 Practice: Logistic Regression and Support Vector Machines

CS201L: Artificial Intelligence Laboratory
Indian Institute of Technology, Dharwad

19 - Feb - 2026

1 Problem Statement

Using the Date Fruit dataset (`DateFruit_Datasets.csv`), implement and compare the performance of Logistic Regression and Support Vector Machines. You are required to categorize the fruit samples into their respective classes based on image-extracted numerical features.

2 Dataset Properties

- **Total Samples:** 898
- **Total Features:** 34 numerical features
- **Target Attribute:** Class (multiclass)
- **Data Splits:** Use the `DateFruit_Train.csv` (60%), `DateFruit_Validation.csv` (20%), and `DateFruit_Test.csv` (20%).

Write a Python program to complete the following tasks.

3 Task 1: Logistic Regression

Perform logistic regression classification and performance metric evaluation as listed below:

1. Load train, validation, and test data from `DateFruit_Train.csv`, `DateFruit_Validation.csv`, and `DateFruit_Test.csv`, respectively.
2. Implement Logistic Regression classification with `sklearn.linear_model.LogisticRegression`
3. Analyze the importance of features by observing the absolute values of the learned weight vector $\mathbf{w}$. Visualize the weight distribution by plotting a bar graph.

3.1 Sample Code for Logistic Regression

```
1 from sklearn.linear_model import LogisticRegression
2
3 # Step 1: Initialize the Logistic Regression model
4 logistic_reg = LogisticRegression(max_iter=1000)
5
6 # Step 2: Model Training (Parameter estimation)
7 # Parameter estimation or model training is performed using the
8   fit function
9 logistic_reg.fit(X_train, y_train)
10
11 # Step 3: Predict class labels for test or validation data
12 # Use the predict function to generate class labels
13 y_test_pred = logistic_reg.predict(X_test)
14
15 # Access the weight vector
16 weights = logistic_reg.coef_
```

4 Task 2: Support Vector Machine with Different Kernels

1. Load train, validation, and test data from `DateFruit_Train.csv`, `DateFruit_Validation.csv`, and `DateFruit_Test.csv`, respectively.
2. Perform the classification using the Support Vector Machine (SVM) method with different kernels: linear, polynomial (degree [2, 3, 4, 5]), and RBF, with different values of C . Train the model on the training data using the built-in SVM function. The syntax for changing the kernels and hyperparameter C is given below. Fine-tune the regularization parameter C using the validation data.
3. After selecting the optimal C , use the trained model to predict the class labels for the validation and test data. Finally, you will have the actual and predicted class labels for both the validation and test data. Generate a weight plot from the model weights. Give the results in the form of table for different values of C and different kernels

4.1 Sample Code for Support Vector Machine (Linear Kernel)

```
1 from sklearn.svm import SVC
2
3 # Step 1: Initialize the SVM model with linear kernel
4 linear_svm = SVC(kernel='linear', C=1.0) # You can adjust the
5   parameter C here
6
7 # Step 2: Model Training
8 # Model training is performed using the fit function
9 linear_svm.fit(X_train, y_train)
```

```

9
10 # Step 3: Predict class labels for test or validation data
11 # Use the predict function to generate class predictions
12 y_test_pred = linear_svm.predict(X_test)
13
14 # Access the weights (only available for linear kernel)
15 weights = linear_svm.coef_

```

Hint: Check the shape of weights produced by SVM.

4.2 Sample Code for Support Vector Machine (Polynomial Kernel)

Experiment with polynomial kernel to capture non-linear relationships in the data.

```

1 from sklearn.svm import SVC
2
3 # Step 1: Initialize the SVM model with polynomial kernel
4 poly_svm = SVC(kernel='poly', degree=3, C=1.0, gamma='scale')
5 # degree: degree of the polynomial (e.g., 2, 3, 4, 5)
6 # C: regularization parameter
7 # gamma: kernel coefficient ('scale', 'auto', or a float value)
8
9 # Step 2: Model Training
10 poly_svm.fit(X_train, y_train)
11
12 # Step 3: Predict class labels for test or validation data
13 y_test_pred = poly_svm.predict(X_test)
14
15 # Note: For polynomial kernel, weights are not directly
16 # accessible
17 # as the decision boundary is in a higher-dimensional space

```

4.3 Sample Code for Support Vector Machine (RBF Kernel)

Experiment with RBF kernel, which is effective for non-linear classification problems.

```

1 from sklearn.svm import SVC
2
3 # Step 1: Initialize the SVM model with RBF kernel
4 rbf_svm = SVC(kernel='rbf', C=1.0, gamma='scale')
5 # C: regularization parameter (controls the trade-off between
6 # smooth decision boundary and classifying training points
7 # correctly)
8 # gamma: kernel coefficient (controls how far the influence of a
9 # single training example reaches)
10 # 'scale' (default): 1 / (n_features * X.var())
11 # 'auto': 1 / n_features
12 # or specify a float value
13
14 # Step 2: Model Training

```

```

14 rbf_svm.fit(X_train, y_train)
15
16 # Step 3: Predict class labels for test or validation data
17 y_test_pred = rbf_svm.predict(X_test)
18
19 # Note: For RBF kernel, weights are not directly accessible
20 # as the decision boundary is in a high-dimensional space

```

4.4 Parameter Tuning Guidelines

- **C (Regularization Parameter):**

- Controls the trade-off between maximizing the margin and minimizing the classification error.
- Small **C**: Larger margin, more misclassifications allowed (underfitting risk)
- Large **C**: Smaller margin, fewer misclassifications (overfitting risk)
- Try values: [0.001, 0.01, 0.1, 1, 10, 100, 1000]

- **gamma (Kernel Coefficient for 'poly' and 'rbf'):**

- Defines how far the influence of a single training example reaches.
- Small **gamma**: Far reach, smoother decision boundary (underfitting risk)
- Large **gamma**: Close reach, more complex decision boundary (overfitting risk)
- Try values: ['scale', 'auto', 0.001, 0.01, 0.1, 1, 10]

- **degree (For polynomial kernel only):**

- Degree of the polynomial kernel function.
- Try values: [2, 3, 4, 5]

5 Important Note on Data Scaling

Important: The Date Fruit dataset contains features with very diverse value ranges. You may face convergence issues or poor performance when training the models. To avoid this:

1. **Scale your data** using standardization or normalization before training.
2. Use `StandardScaler` from `sklearn.preprocessing` to scale features to have zero mean and unit variance.
3. Apply the same scaling transformation to train, validation, and test data.
4. Scaling is particularly important for SVM with RBF and polynomial kernels, as well as for Logistic Regression.

5.1 Sample Code for Data Scaling

```
1 from sklearn.preprocessing import StandardScaler
2
3 # Step 1: Initialize the scaler
4 scaler = StandardScaler()
5
6 # Step 2: Fit the scaler on training data and transform
7 X_train_scaled = scaler.fit_transform(X_train)
8
9 # Step 3: Transform validation and test data using the same
10 # scaler
11 # IMPORTANT: Do NOT fit on validation/test data, only transform
12 X_validation_scaled = scaler.transform(X_validation)
13 X_test_scaled = scaler.transform(X_test)
14
15 # Now use X_train_scaled, X_validation_scaled, and X_test_scaled
16 # for training and evaluation
```

Why Scaling Matters:

- Features with larger ranges can dominate the learning process.
- SVM and Logistic Regression are sensitive to feature scales.
- Scaling ensures all features contribute equally to the model.
- Improves convergence speed and model performance.

6 Task 3: Performance Metrics Evaluation

Evaluate the performance of both Logistic Regression and SVMs (with different kernels) on the validation and test data. Report (print) the following metrics:

1. Confusion Matrix
2. Accuracy
3. Precision
4. Recall
5. F1-score

Use `micro` for overall and `macro`-averaging for multiclass evaluation.

6.1 Sample Code for Performance Evaluation

```

1 from sklearn.metrics import confusion_matrix, accuracy_score
2 from sklearn.metrics import precision_score, recall_score,
   f1_score
3
4 # Calculate metrics
5 conf_matrix = confusion_matrix(y_test, y_test_pred)
6 accuracy = accuracy_score(y_test, y_test_pred)
7 precision_macro = precision_score(y_test, y_test_pred, average='
   macro')
8 precision_micro = precision_score(y_test, y_test_pred, average='
   micro')
9 recall_macro = recall_score(y_test, y_test_pred, average='macro')
10 recall_micro = recall_score(y_test, y_test_pred, average='micro')
11 f1_macro = f1_score(y_test, y_test_pred, average='macro')
12 f1_micro = f1_score(y_test, y_test_pred, average='micro')
13
14 # Print results
15 print("Confusion Matrix:\n", conf_matrix)
16 print(f"Accuracy: {accuracy:.4f}")
17 print(f"Precision (Macro): {precision_macro:.4f}")
18 print(f"Precision (Micro): {precision_micro:.4f}")
19 print(f"Recall (Macro): {recall_macro:.4f}")
20 print(f"Recall (Micro): {recall_micro:.4f}")
21 print(f"F1-Score (Macro): {f1_macro:.4f}")
22 print(f"F1-Score (Micro): {f1_micro:.4f}")

```